Hermès — moteur de contrôles générique et audit ligne à ligne

Note de conception issue de l'échange du 6 novembre 2026. Elle propose un modèle pour passer d'une quarantaine de contrôles codés un par un à des contrôles **déclarés**, exécutés par un moteur commun, avec un audit qui permet de suivre chaque ligne de données à chaque étape.

Principes

- **Aucun appel à un modèle dans le calcul d'un contrôle**, sauf dans une étape « IA » explicitement déclarée, dont les propositions sont validées par un humain.
- **Les définitions sont de la configuration** (JSON versionné dans Git) ; **les exécutions sont des données** (base, ou Parquet interrogé avec DuckDB).
- **On n'écrase jamais rien** : relancer un calcul crée une nouvelle exécution.
- **Chaque décision dit sur quoi elle s'appuie** : règle, écritures historiques ou raisonnement de l'IA.

Deux mondes : la recette et le carnet

	Définitions (la recette)	Exécution (le carnet)
Contenu	contrôles, pipelines, règles	exécutions, évaluations, résultats, validations
Rythme	change rarement, relu en merge request	écrit à chaque passage, gros volume
Stockage	JSON dans Git, validé par un schéma Pydantic	base de données ou Parquet + DuckDB
Source de vérité	le fichier	la table

Le pont entre les deux est le **manifeste d'exécution** : au lancement, le moteur charge la définition, la valide, calcule son empreinte et l'enregistre avec l'exécution. Une copie des règles peut être importée en base, en lecture seule, pour l'interface ou les requêtes, marquée de la même empreinte.

Modèle de données

Définitions (JSON)

- **Contrôle** : identifiant, nom, description métier.
- **Définition de contrôle**, versionnée : pipeline = liste ordonnée d'étapes, chacune d'un **type** (`regles` , `historique` , `ia` , demain `classifieur`), avec ses paramètres.
- **Jeu de règles**, par contrôle et versionné : chaque règle a un identifiant stable, ses conditions et la catégorie qu'elle attribue.

Exécution (tables)

`execution` — une ligne par exécution.

Colonne	Rôle
`execution_id`	identifiant unique
`controle_id` , `definition_version`	contrôle et version du pipeline suivi
`regles_empreinte` , `code_version`	empreinte du jeu de règles, commit du code
`entree_empreinte`	empreinte SHA-256 du fichier Excel lu
`demarree_le` , `terminee_le` , `statut`	horodatage et issue
`remplace_execution_id`	exécution que celle-ci remplace, s'il y en a une

`ligne_donnee` — une ligne par ligne du fichier d'entrée : `execution_id` , `row_uid` (attribué à la lecture, propagé partout), numéro de ligne dans le fichier, valeurs d'origine.

`evaluation` — la table mère : une ligne par évaluation d'une étape (et, pour les règles, d'une règle) sur une ligne de données.

Colonne	Rôle
`evaluation_id`	identifiant unique
`execution_id` , `row_uid`	exécution et ligne évaluées
`etape_rang` , `etape_type`	position de l'étape dans le pipeline et son type
`rang`	ordre d'évaluation dans l'étape (règles testées l'une après l'autre)
`statut`	`appliquee` ou `non_applicable`

Colonne	Rôle
`macro_proposee`, `sous_macro_proposee`	catégorie proposée, si l'évaluation a joué

Tables filles, une par type d'étape, reliées par `evaluation_id` (la fille pointe vers la mère) :

- **`evaluation_regle`** : `regle_id`, `regle_version`, `condition_echouee` (la première condition fausse, pour comprendre pourquoi une règle n'a pas joué), valeurs avant et après pour la règle appliquée seulement.
- **`evaluation_historique`** : critère de rapprochement, nombre d'écritures passées retrouvées, identifiants de ces écritures.
- **`evaluation_ia`** : modèle, version du prompt, confiance, explication, identifiant de l'appel à la passerelle.

`resultat` — l'arrivée : une ligne par ligne de données et par exécution, avec la macro et la sous-macro retenues, l'étape et l'évaluation qui ont décidé, la confiance et l'explication pour l'IA, puis la validation humaine (qui, quand, catégorie corrigée).

Pourquoi une table mère et des tables filles

- La table mère reste simple, sans colonnes toujours vides.
- Chaque détail appartient forcément à une évaluation existante.
- Un nouveau type d'étape = une nouvelle table fille, sans toucher au reste.

Le modèle respecte la troisième forme normale : chaque colonne dépend de la clé, de toute la clé et de rien d'autre. La découpe en tables filles est un motif de modélisation (une table par sous-type), pas une forme normale.

Volume

Tracer toutes les règles testées, pas seulement la gagnante : environ 16 000 lignes × une centaine de règles ≈ 1,6 million d'évaluations par exécution, ce que Parquet et DuckDB tiennent sans difficulté. Le journal est rangé par exécution, pour purger selon la durée de conservation.

Lire le parcours d'une ligne

Filtrer `evaluation` sur `execution_id` et `row_uid`, trier par `etape_rang` puis `rang` : on voit chaque étape et chaque règle, dans l'ordre. On n'ouvre une table fille que pour le détail.

Passer à l'échelle sur une quarantaine de contrôles

- Chaque **type d'étape** est codé une fois dans le moteur, avec sa table fille, et servi à tous les contrôles.
- Un contrôle n'a que les étapes dont il a besoin : règles seules, règles puis historique, ou les trois.
- **Ajouter un contrôle** = écrire sa définition et ses règles, les relire en merge request, pas recoder un pipeline.
- Un **agent de conception** peut proposer la définition et les jeux de test à partir de la description métier ; un humain valide ; le moteur exécute sans IA.

Catégorisation des écritures manuelles

Aujourd'hui, environ 7 000 écritures sur 16 000 arrivent à l'IA, surtout pour trouver la **sous-macro** d'une macro déjà connue.

1. **Mesurer** : répartir ces 7 000 par macro et compter l'historique validé de chacune.
2. **Classifieur classique par macro**, entraîné sur l'historique validé : choix entre quelques sous-macros, rapide, quasi gratuit, avec un score de confiance.
3. **IA ou agent seulement quand le classifieur hésite**, avec l'explication.
4. **Agent qui propose de nouvelles règles de mapping** à partir des choix validés : la première passe grossit à chaque arrêté.
5. **Nouvelle sous-macro** : ajoutée au référentiel versionné avec sa description ; l'IA la prend en charge tout de suite ; réentraînement et règles dès qu'il y a assez d'exemples validés. Une chute de confiance sur une macro signale souvent une catégorie ou une pratique nouvelle.

Première brique : construction et enrichissement du fichier

Le contrôle des écritures manuelles a deux briques : la construction du fichier, puis sa catégorisation. La première suit le même modèle d'audit que la seconde.

1. **Construction** : base Matisse, enrichie par eMatch et d'autres bases (flux EasyMorph réimplémenté en Python). Chaque ligne garde son identifiant Matisse ; on trace, par colonne, la source et la clé de jointure.
2. **Checks** : chaque vérification (valeurs manquantes, etc.) est une règle. Une ligne `evaluation` par ligne et par check : réussi ou échoué, avec la condition non respectée.

3. **Corrections du métier** : une table `correction` (ligne, colonne, ancienne valeur, nouvelle valeur, auteur, date, motif, check à l'origine). Le métier supprime des lignes, modifie des valeurs, remplit des cases vides ; il n'ajoute pas de colonnes.
4. **Catégorisation** : reçoit le fichier construit + les corrections appliquées, plus jamais un fichier retouché à la main sans trace.

Invariants vérifiables sans EasyMorph : nombre de lignes Matisse conservé après chaque jointure, pas de doublon créé, totaux des montants identiques.

Comparer malgré les retouches : jointure sur l'identifiant Matisse entre la sortie Python et le fichier retouché, sur les seules colonnes produites par la pipeline. Trois catégories d'écarts :

Écart	Lecture
Ligne absente du fichier retouché	suppression du métier
Case vide chez nous, remplie chez lui	ajout du métier
Valeur différente	correction ou bug

Un écart sur une ligne qui a échoué un check est probablement une correction ; sur une ligne qui a tout réussi, c'est probablement un bug. Beaucoup d'écarts sur une même colonne avec la même logique : regarder le code d'abord. Le métier valide un échantillon des valeurs différentes.

Bonus : on voit quels checks déclenchent le plus de corrections, et lesquelles pourraient être automatisées.

Équivalence avec EasyMorph

Les données disponibles ont été retouchées par le métier après EasyMorph : un écart ne dit pas s'il s'agit d'un bug ou d'une correction.

- **Jeu de référence** : la personne qui a EasyMorph et les accès gèle les entrées brutes d'un arrêté, lance le projet, exporte la sortie et les tables intermédiaires de chaque passe. Le Python tourne sur les mêmes entrées ; comparaison ligne par ligne par `row_uid`, passe par passe.
- **Sans accès** : extraire les règles du fichier EasyMorph et fabriquer des jeux de test fictifs, règle par règle, que le métier passe dans EasyMorph.
- **Tri des écarts** : un écart sur une colonne que le métier ne touche jamais est probablement un bug.

Chatbot d'audit

Les contrôleurs interrogent les exécutions en langage naturel. L'agent **ne calcule pas et n'écrit pas de SQL libre** : il appelle des outils typés, en lecture seule, puis formule la réponse en citant les identifiants d'exécution, de ligne et de règle, pour qu'on puisse vérifier.

Outil	Entrée	Sortie
`bilan_execution`	exécution, macro facultative	écritures décidées par étape, en distinguant ce qui a été attribué : macro, sous-macro ou les deux
`trouver_ligne`	exécution + clé métier, ou valeurs de colonnes, ou numéro de ligne du fichier	la ou les lignes correspondantes
`parcours_ligne`	exécution, ligne	chaque étape et chaque règle testée, dans l'ordre, avec la condition échouée
`lignes_par_regle`	exécution, règle	lignes où la règle a joué
`decisions_ia`	exécution, seuil de confiance	propositions de l'IA sous le seuil, ou corrigées par un humain
`comparer_executions`	deux exécutions	lignes qui ont changé de catégorie, et la règle en cause

Retrouver une ligne par ce que voit le contrôleur. Il n'a pas le `row_uid` : il a ses valeurs.

1. **Clé métier** déclarée dans la définition du contrôle et stockée avec chaque ligne : la recherche est exacte.
2. **Numéro de ligne** dans le fichier Excel d'origine.
3. **Valeurs de colonnes**, en secours : seules les colonnes déclarées dans la définition sont acceptées ; espaces, majuscules, montants et dates sont normalisés. Plusieurs correspondances → l'agent les montre et demande de préciser.

À ajouter au modèle : dans `evaluation`, la liste des champs remplis par l'évaluation (`macro`, `sous_macro`), pour que le bilan distingue les deux ; dans `ligne_donnee`, la clé métier.

Types d'étapes, stratégies et briques partagées

D'un contrôle à l'autre, une même étape ne travaille pas de la même façon. On sépare donc trois niveaux.

1. **Le type d'étape est un contrat** : il reçoit les lignes encore non décidées et le contexte du contrôle, il rend une décision par ligne avec ses preuves. Le moteur ne connaît que ce contrat.
2. **Sous chaque type, plusieurs stratégies**, chacune une classe qui respecte le contrat, enregistrée par son nom dans un registre, avec son propre schéma Pydantic de paramètres.
3. **Des briques partagées en dessous** : modèles de machine learning (versionnés dans MLflow), référentiels de catégories, connecteurs de données. Une stratégie les référence par nom et version.

Type	Stratégies possibles
`regles`	première règle applicable, règles par priorité
`historique`	clé de réconciliation, modèle de machine learning qui cherche un motif
`ia`	prompt avec la connaissance métier, agent avec outils (dont l'appel à un modèle de machine learning)

La définition d'un contrôle déclare, pour chaque étape : son type, sa stratégie et ses paramètres. Par exemple : étape 2, `historique`, stratégie `cle_reconciliation`, clé = journal + compte + libellé. Le moteur trouve la stratégie dans le registre et valide ses paramètres au chargement.

Pour l'audit, chaque évaluation note en plus la stratégie et sa version ; le détail propre à la stratégie va dans la table fille de son type.

Ajouter un contrôle, c'est assembler des stratégies existantes ; on ne développe que les stratégies nouvelles, qui resservent ensuite partout.

Deep Agents ou agent codé à la main

Plus la tâche est ouverte, plus un cadre comme Deep Agents vaut le coup ; plus elle est cadrée, plus un graphe LangGraph codé à la main est préférable.

Ce que Deep Agents apporte tout fait : la planification (l'agent tient et met à jour sa liste de tâches), un système de fichiers de travail pour les notes et les documents longs, des sous-agents à qui déléguer une sous-tâche dans un contexte propre, et des middlewares pour brancher les contrôles.

Agent	Nature de la tâche	Choix
Conception d'un contrôle	longue et ouverte : lire la spec métier et le fichier EasyMorph, explorer des données d'exemple, rédiger le JSON, générer les tests, corriger	**Deep Agents**
Audit d'une exécution	courte et cadrée : une question, quelques outils typés en lecture seule	**graphe codé à la main** : plus prévisible, plus rapide, moins cher, plus facile à auditer

Plan de mise en place en deux jours

Principe : ne rien réécrire, faire évoluer l'existant sur le seul contrôle des écritures manuelles. Chaque étape est utile seule et ne change aucun résultat.

Jour 1 — l'audit

- [ ] Matin : figer 2 ou 3 fichiers d'entrée et leurs sorties actuelles comme référence (test de non-régression).
- [ ] Matin : tables `execution`, `ligne_donnee` (avec la clé métier), `evaluation` et ses trois tables filles, `resultat`, en Parquet + DuckDB.
- [ ] Après-midi : faire écrire une trace au code actuel à chaque règle testée, à l'historique et à l'IA, sans toucher au calcul.
- [ ] Fin de journée : rejouer les fichiers de référence, sorties identiques, et lire le parcours de quelques lignes.

Jour 2 — le contrat d'étape et la démonstration

- [ ] Matin : emballer le code existant en trois stratégies (règles, historique, IA) derrière le contrat d'étape, avec un registre.
- [ ] Matin : définition JSON du contrôle, validée par Pydantic ; le moteur l'exécute ; sorties identiques à la référence.
- [ ] Après-midi : deux outils d'audit, `bilan_execution` et `trouver_ligne` + `parcours_ligne`, branchés sur le chatbot existant.
- [ ] Fin de journée : démonstration sur un arrêté.

Après les deux jours : contrôle 002 sur le moteur (preuve de généricité), autres outils d'audit, agent de conception de contrôles, classifieur des sous-macros.

Pour aller vite : confier chaque tâche aux agents de construction du kit Hermès, une tâche par session, test de référence relancé à chaque fois.

Prochaines étapes

1. Demander au métier un jeu de référence EasyMorph sur un arrêté.
2. Mesurer la répartition des 7 000 écritures et l'historique validé par macro.
3. Écrire le schéma Pydantic des définitions et les tables d'exécution, sur le contrôle des écritures manuelles.
4. Prototyper le classifieur sur la macro la plus fréquente.